

Centralized Logging on Kubernetes

Fluent Bit · Elasticsearch · Kibana

A step-by-step implementation guide for deploying a centralized log collection pipeline on a Kubernetes cluster using Fluent Bit as a DaemonSet collector, Elasticsearch as the storage and indexing backend, and Kibana as the search interface. The guide covers every resource created, all commands run, the reasoning behind each decision, blockers encountered, and how they were resolved.

Sensitive values — passwords, tokens, encryption keys — are replaced with descriptive placeholders throughout.

Contents

1	Environment and Prerequisites
2	Cluster and Namespace Preparation
3	Elasticsearch — Deployment
4	Elasticsearch — Post-Start Configuration
5	Kibana — Deployment
6	Kibana — Access and Verification
7	Fluent Bit — Deployment
8	Verifying the Full Pipeline
9	Kibana — Data View and Saved Search
10	Troubleshooting Log
11	Quick Reference
12	Known Gaps and Next Steps

1. Environment and Prerequisites

1.1 Environment

- **OS:** Windows with Hyper-V
- **Kubernetes:** Rancher Desktop (single-node local cluster)
- **Container runtime:** containerd
- **kubectl:** bundled with Rancher Desktop, available in PATH after install
- **Shell used for commands:** PowerShell / Windows Terminal
- **Application:** Spring Boot PetClinic, running in the *petclinic* namespace, plain-text stdout logs

1.2 Stack Versions

- Elasticsearch: **8.19.12** (official image from docker.elastic.co)
- Kibana: **8.19.12** (must exactly match Elasticsearch)
- Fluent Bit: **3.0.7** (from cr.fluentbit.io/fluent/fluent-bit)

1.3 TLS Decision — What Was Used vs What Was Discussed

Elasticsearch 8.x ships with HTTPS and mutual TLS enabled by default. During the session, HTTPS, CA certificate extraction, and Kibana TLS configuration were explored in detail. However, those paths introduced complexity and certificate SAN mismatches that could not be resolved quickly in the lab window.

The final working setup uses **plain HTTP** on port 9200 — TLS is disabled on the Elasticsearch HTTP layer. All three components (Elasticsearch, Kibana, Fluent Bit) communicate over HTTP inside the cluster. Basic authentication (username/password) is still active, so connections are authenticated even without encryption.

Important

This is suitable for a local lab environment. In production, TLS should be re-enabled and CA certificates properly distributed. See Section 12 for the upgrade path.

2. Cluster and Namespace Preparation

2.1 Verify the Cluster is Reachable

Before deploying anything, confirm `kubectl` can reach the Rancher Desktop cluster:

```
kubectl cluster-info  
kubectl get nodes
```

Expected: one node in Ready state. Rancher Desktop runs everything in a single VM.

2.2 Create the Logging Namespace

All logging infrastructure lives in a dedicated namespace. This keeps it cleanly separated from the application namespace and makes RBAC and cleanup straightforward. The namespace is defined in a YAML manifest so it is version-controlled alongside the rest of the stack.

```
kubectl apply -f 01-namespace.yml
```

2.3 Manifest File Strategy

Each Kubernetes resource is defined in a separate YAML file, numbered to reflect the intended apply order. The files are applied with *kubectl apply -f*. The numbering convention used:

- **01-namespace.yml** — Logging namespace
- **02-statefulset.yml** — Elasticsearch StatefulSet
- **03-service.yml** — Elasticsearch ClusterIP Service
- **04-kibana-deploy.yml** — Kibana Deployment
- **05-kibana-configmap.yml** — Kibana configuration (kibana.yml)
- **06-kibana-service.yml** — Kibana ClusterIP Service
- **07-fluentbit-sa.yml** — Fluent Bit ServiceAccount
- **08-fluentbit-crole.yml** — Fluent Bit ClusterRole
- **09-fluentbit-crolebind.yml** — Fluent Bit ClusterRoleBinding
- **10-fluentbit-configmap.yml** — Fluent Bit configuration
- **11-fluentbit-daemonset.yml** — Fluent Bit DaemonSet

Secrets are created imperatively (via `kubectl` commands) rather than in YAML files to avoid storing credential values in version control.

3. Elasticsearch — Deployment

3.1 What the StatefulSet Does

Elasticsearch is stateful — it keeps index data on disk across restarts. A StatefulSet gives the pod a stable, predictable hostname (**elastic-logging-0**) and a dedicated PersistentVolumeClaim that survives pod restarts. A regular Deployment would not guarantee hostname stability or PVC identity.

3.2 Key Configuration Choices

- **discovery.type=single-node** — disables cluster formation. Without this, a single-node Elasticsearch waits indefinitely for other nodes to form a quorum and never starts.
- **network.host=0.0.0.0** — binds to all interfaces inside the pod, not just loopback. Required so the Service can forward traffic to the pod.
- **ES_JAVA_OPTS=-Xms256m -Xmx256m** — pins the JVM heap to 256 MB. Without this, Elasticsearch tries to allocate a large fraction of available RAM and the pod gets OOMKilled on a resource-constrained local cluster.
- **xpack.security.http.ssl.enabled=false** — disables TLS on the HTTP layer. This is set explicitly because ES 8.x enables TLS by default. See Section 1.3 for the reasoning.
- **xpack.security.enabled=true** — keeps authentication active even without TLS. Username/password is still required for all API calls.

3.3 Storage

The StatefulSet uses a *volumeClaimTemplate* to request a 1 Gi PersistentVolumeClaim per pod. The StorageClass is **local-path**, which is the default in Rancher Desktop and provisions host-path volumes automatically. The resulting PVC name is **data-elastic-logging-0**.

3.4 Service

A ClusterIP Service named **elasticsearch** is created in the logging namespace on port 9200. Other pods in the cluster reach Elasticsearch at the DNS name *elasticsearch.logging.svc*. The Service selects pods with the label *app: elasticsearch*.

3.5 Applying the Manifests

```
kubectl apply -f 02-statefulset.yml -n logging
kubectl apply -f 03-service.yml -n logging
```

3.6 Waiting for the Pod to Be Ready

Elasticsearch takes 30–60 seconds to initialize on first start. Watch the pod status and check the logs:

```
kubect1 get pods -n logging -w
# Wait until elastic-logging-0 shows Running / 1/1

kubect1 logs -n logging elastic-logging-0 --tail=40
```

Verification

Look for: "Cluster health status changed from [RED] to [GREEN]" and "started" in the log output. The pod label shows 1/1 READY.

3.7 Verify PVC Bound

```
kubect1 get pvc -n logging
```

Expected: *data-elastic-logging-0* shows status **Bound**.

4. Elasticsearch — Post-Start Configuration

After Elasticsearch is running, several configuration steps must be done via its API before Kibana and Fluent Bit can connect. These steps are performed using curl through a kubectl port-forward to the Elasticsearch pod.

4.1 Open a Port-Forward to Elasticsearch

```
# Run in a separate terminal and leave it open
kubectl -n logging port-forward svc/elasticsearch 9200:9200
```

4.2 Test Connectivity

Confirm the API is reachable and authentication is working using the built-in *elastic* superuser. The password is auto-generated on first start and printed in the pod logs.

```
# Get the auto-generated password from pod logs (first start only)
kubectl logs -n logging elastic-logging-0 | grep 'Password for the elastic user'

# Test the API
curl -u "elastic:<AUTO_GENERATED_PASSWORD>" http://localhost:9200
```

A JSON response with *cluster_name* and *tagline* confirms the API is reachable. The elastic superuser password is then stored in a Kubernetes Secret for future use:

```
kubectl create secret generic es-auth \
-n logging \
--from-literal=username=elastic \
--from-literal=password="<ES_SUPERUSER_PASSWORD>"
```

4.3 Create the Fluent Bit Role

A dedicated role is created with the minimum permissions Fluent Bit needs to write logs. It cannot read data, manage indices, or access any other resource.

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
-X POST http://localhost:9200/_security/role/fluentbit_writer \
-H 'Content-Type: application/json' \
-d '{
  "cluster": ["monitor"],
  "indices": [{
    "names": ["petclinic-*"],
    "privileges": ["auto_configure", "create_index", "write"]
  }]
}'
```

The role permissions explained:

- **monitor** (cluster) — Fluent Bit's ES output plugin pings the cluster health endpoint before sending. Without this, the plugin fails to initialize.

- **auto_configure** (index) — allows the plugin to apply index templates automatically.
- **create_index** — allows creation of new daily indices (*petclinic-YYYY.MM.DD*).
- **write** — allows indexing documents.

4.4 Create the Fluent Bit User

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
-X POST http://localhost:9200/_security/user/fluentbit \
-H 'Content-Type: application/json' \
-d '{
  "password": "<FLUENTBIT_PASSWORD>",
  "roles": ["fluentbit_writer"]
}'
```

Store the Fluent Bit credentials in a Kubernetes Secret:

```
kubectl create secret generic fluentbit-es-auth \
-n logging \
--from-literal=username=fluentbit \
--from-literal=password="<FLUENTBIT_PASSWORD>"
```

4.5 Create the Kibana Service Account Token

Kibana authenticates to Elasticsearch using a service account token rather than a username/password. Service account tokens are the recommended credential type for built-in Elastic services — they can be revoked independently and are pre-authorized with the correct permissions for Kibana's system index operations.

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
-X POST
'http://localhost:9200/_security/service/elastic/kibana/credential/token/kibana-token'
```

The response contains a *token.value* field. Copy that value and store it:

```
kubectl create secret generic kibana-es-token \
-n logging \
--from-literal=token="<SERVICE_ACCOUNT_TOKEN_VALUE>"
```

4.6 Generate Kibana Encryption Keys

Kibana requires three independent 32-character encryption keys for saved objects, reporting, and security cookies. Without these, Kibana logs a warning on every start and cannot persist saved searches across restarts. Generate each key with:

```
openssl rand -hex 16 # run this command three separate times
```

Store the three values in a single Secret:

```
kubectl create secret generic kibana-enc-keys \
-n logging \
--from-literal=xpack_encryptedSavedObjects_encryptionKey="<KEY_1>" \
--from-literal=xpack_reporting_encryptionKey="<KEY_2>" \
--from-literal=xpack_security_encryptionKey="<KEY_3>"
```

4.7 Verify All Secrets Exist

```
kubectl get secrets -n logging
```

Expected to see: *es-auth*, *fluentbit-es-auth*, *kibana-es-token*, *kibana-enc-keys*.

5. Kibana — Deployment

5.1 ConfigMap — kibana.yml

Kibana's configuration is provided via a ConfigMap (**05-kibana-configmap.yml**) mounted as a file at `/usr/share/kibana/config/kibana.yml` using a `subPath` mount. The `subPath` mount is necessary to replace only the `kibana.yml` file inside the config directory without overwriting other files Kibana places there at startup. The ConfigMap manifest is applied after the Deployment manifest (**04-kibana-deploy.yml**) because the Deployment spec references it — Kubernetes will reconcile the mount once both exist.

Key settings in `kibana.yml`:

- **server.host: 0.0.0.0** — binds the Kibana HTTP server to all interfaces. Without this, Kibana only listens on localhost inside the pod and the Kubernetes Service cannot forward traffic to it.
- **elasticsearch.hosts** — set to the HTTP URL of the Elasticsearch pod (`http://elastic-logging-0:9200`). The value is injected via an environment variable to keep the ConfigMap generic.
- **elasticsearch.serviceAccountToken** — the Kibana service account token injected from the `kibana-es-token` Secret.
- **xpack.encryptedSavedObjects.encryptedKey**, **xpack.reporting.encryptedKey**, **xpack.security.encryptedKey** — injected from the `kibana-enc-keys` Secret.

Why subPath for the ConfigMap mount?

Mounting a ConfigMap at a directory path replaces the entire directory. Kibana writes its own files into `/usr/share/kibana/config` at startup. Using `subPath: kibana.yml` mounts only the single file, leaving the rest of the directory intact.

5.2 Deployment Environment Variables

The Kibana Deployment injects the following environment variables:

- **ELASTICSEARCH_HOSTS** — sourced from a literal value in the Deployment spec, set to `["http://elastic-logging-0:9200"]`.
- **ELASTICSEARCH_SERVICEACCOUNTTOKEN** — sourced from the `kibana-es-token` Secret, key `token`.
- **XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY** — from `kibana-enc-keys`.
- **XPACK_REPORTING_ENCRYPTIONKEY** — from `kibana-enc-keys`.
- **XPACK_SECURITY_ENCRYPTIONKEY** — from `kibana-enc-keys`.

Using environment variables for these values means the ConfigMap does not need to contain any credentials. The ConfigMap can be version-controlled; the Secrets cannot.

5.3 Applying the Manifests

```
kubectl apply -f 04-kibana-deploy.yml -n logging
kubectl apply -f 05-kibana-configmap.yml -n logging
kubectl apply -f 06-kibana-service.yml -n logging
```

5.4 Watching the Rollout

```
kubectl rollout status deploy/kibana -n logging
# OR watch manually:
kubectl get pods -n logging -w
```

Kibana takes 60–90 seconds to connect to Elasticsearch and finish startup. If the pod restarts repeatedly, check the logs:

```
kubectl logs -n logging deploy/kibana --tail=50
```

6. Kibana — Access and Verification

6.1 The Windows Port Exclusion Problem

On Windows with Hyper-V enabled, the OS reserves certain dynamic port ranges for its own use. Port 5601 — Kibana's default port — falls within one of these excluded ranges. Attempting to run *kubect/port-forward ... 5601:5601* fails silently or with a binding error.

To diagnose, list the excluded ranges:

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

If 5601 appears in any listed range, choose a local port that does not appear in the output. Port 9300 works in this environment:

```
# Run in a separate terminal and leave it open
kubectl -n logging port-forward svc/kibana 9300:5601
```

Kibana is then accessible in a browser at **<http://localhost:9300>**.

6.2 Login

Log in with the *elastic* superuser credentials (username: *elastic*, password: the value stored in the **es-auth** Secret).

6.3 Verify Kibana is Connected

Before opening the browser, confirm Kibana has finished starting and connected to Elasticsearch via its status API:

```
curl http://localhost:9300/api/status
```

Verification

The response should contain "level": "available" under status.overall. Any other level (degraded, unavailable) means Kibana has not yet connected to Elasticsearch — wait a few more seconds and retry.

7. Fluent Bit — Deployment

7.1 Why a DaemonSet?

Container log files live on the host filesystem of each cluster node, under `/var/log/containers/`. A DaemonSet ensures exactly one Fluent Bit pod runs on every node, so no node's logs are missed. When Rancher Desktop adds a node, Kubernetes automatically schedules a Fluent Bit pod on it.

7.2 RBAC Setup

The Kubernetes metadata enrichment filter (step 7.6) contacts the Kubernetes API to look up pod details. This requires API permissions. Three resources are created:

- **ServiceAccount (fluentbit)** in the logging namespace — the identity under which Fluent Bit pods run.
- **ClusterRole (fluentbit)** — grants *get*, *list*, *watch* on *pods* and *namespaces* cluster-wide. Only read access, on only these two resource types.
- **ClusterRoleBinding (fluentbit)** — binds the ClusterRole to the ServiceAccount in the logging namespace.

```
kubectl apply -f 07-fluentbit-sa.yml -n logging
kubectl apply -f 08-fluentbit-crole.yml
kubectl apply -f 09-fluentbit-crolebind.yml
```

7.3 Host Volume Mounts

The DaemonSet mounts two host paths into each Fluent Bit pod:

- **/var/log** — contains the *containers/* subdirectory with symlinks to container log files. The Tail input plugin reads from `/var/log/containers/*.log`.
- **/var/lib/docker/containers** — the real log file storage location that the symlinks point to. This mount is needed so Fluent Bit can follow symlinks through to the actual file content.

7.4 The CRI Parser

containerd (the runtime used by Rancher Desktop) writes each log line in CRI format:

```
2024-01-15T10:23:45.123456789Z stdout F <actual log message>
```

Without a parser that understands this format, Fluent Bit treats the entire raw line — timestamp, stream, flag, and message — as a single unstructured string. The CRI parser splits it into proper fields (*time*, *stream*, *logtag*, *log*), making the actual message available as the *log* field in Elasticsearch.

The parser is defined in `parsers.conf` inside the ConfigMap and referenced by the Tail input via *Parser cri*.

7.5 INPUT — Tail Configuration

- **Path `/var/log/containers/*.log`** — watches all container log files on the node.

- **Tag kube.*** — applies a tag starting with *kube.* to every event. This tag is used by downstream filters and outputs to match only these events.
- **Parser cri** — applies the CRI parser defined in `parsers.conf`.
- **Mem_Buf_Limit 10MB** — limits how much memory Fluent Bit uses to buffer events when the downstream is slow.
- **Skip_Long_Lines On** — drops lines exceeding the buffer limit rather than crashing. Prevents issues with unexpectedly large log lines.

7.6 FILTER — Kubernetes Metadata Enrichment

The kubernetes filter uses the Kubernetes API to look up metadata for each log event, using the pod name and namespace extracted from the log file path. It appends the following fields to every event:

- `kubernetes.pod_name`
- `kubernetes.namespace_name`
- `kubernetes.container_name`
- `kubernetes.container_image`
- `kubernetes.labels.*`

Setting **Merge_Log On** tells the filter to parse the *log* field as JSON if possible. Spring Boot logs are plain text, so this has no effect here, but it enables automatic JSON parsing for apps that emit structured logs. **Keep_Log Off** removes the raw *log* field after merging to avoid duplication.

7.7 OUTPUT — Elasticsearch Configuration

- **Host elasticsearch.logging.svc** — the ClusterIP Service DNS name. Port 9200.
- **HTTP_User / HTTP_Passwd** — the Fluent Bit writer credentials, injected from the *fluentbit-es-auth* Secret as environment variables `ES_USER` and `ES_PASS`.
- **Logstash_Format On** — creates daily indices named `<prefix>-YYYY.MM.DD`.
- **Logstash_Prefix petclinic** — daily index names follow *petclinic*-`YYYY.MM.DD`.
- **Suppress_Type_Name On** — required for Elasticsearch 8.x. Older versions used document types; 8.x removed them. Sending a type field causes an error.
- **Replace_Dots On** — replaces dots in field names with underscores to prevent Elasticsearch from interpreting them as nested object paths, which would cause mapping conflicts.
- **Retry_Limit False** — retries failed sends indefinitely. Prevents log loss if Elasticsearch is temporarily unavailable.

7.8 Applying the ConfigMap and DaemonSet

```
kubectl apply -f 10-fluentbit-configmap.yml -n logging
kubectl apply -f 11-fluentbit-daemonset.yml -n logging
```

7.9 Verify Pods Are Running

```
kubectl get pods -n logging
# Should show one fluentbit-<hash> pod per node, status Running

kubectl logs -n logging -l app=fluentbit --tail=50
```

Verification

In the Fluent Bit logs, look for lines like: "[output:es] elastic-logging... HTTP status=200". The absence of errors and the presence of 200 responses confirms data is flowing into Elasticsearch.

7.10 Log Level

During troubleshooting, Fluent Bit's log level was set to *debug* in the ConfigMap's *[SERVICE]* block. Once the pipeline was confirmed working, it was changed back to *info* to reduce noise. After editing the ConfigMap, restart the DaemonSet to pick up the change:

```
kubectl rollout restart -n logging ds/fluentbit
kubectl rollout status -n logging ds/fluentbit
```

8. Verifying the Full Pipeline

8.1 Check That Indices Exist

With the Elasticsearch port-forward active (Section 4.1):

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
"http://localhost:9200/_cat/indices?v"
```

Expected: one or more indices matching *petclinic-YYYY.MM.DD* with a non-zero docs.count.

8.2 Count Documents

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
"http://localhost:9200/petclinic-*/_count?pretty"
```

A *count* greater than zero confirms documents are being indexed. During the session this exceeded 10,000 documents — the DaemonSet was ingesting logs from all namespaces in the cluster, including Fluent Bit's own logs.

8.3 Inspect a Sample Document

```
curl -u "elastic:<ES_SUPERUSER_PASSWORD>" \
"http://localhost:9200/petclinic-*/_search?size=1&pretty"
```

Verify the document contains the expected fields: *@timestamp*, *log* (the message), *kubernetes.pod_name*, *kubernetes.namespace_name*, *kubernetes.container_image*.

8.4 Summary of What Is Confirmed Working

- Fluent Bit is running on the node, tailing container logs.
- The CRI parser is correctly splitting log lines into fields.
- The Kubernetes metadata filter is enriching events with pod/namespace data.
- The Elasticsearch output plugin is successfully authenticating and indexing.
- Daily indices are being created automatically.
- Kibana can reach Elasticsearch and its status is *available*.

9. Kibana — Data View and Saved Search

9.1 Creating the Data View

A Data View tells Kibana which Elasticsearch indices to query and which field represents time. Without it, none of the Discover, Dashboard, or Alerting features work.

- Open Kibana at `http://localhost:9300`. Log in as elastic.
- Go to **Stack Management** → **Data Views** → **Create data view**.
- Set **Index pattern** to `petclinic-*` — this matches all daily indices.
- Set **Timestamp field** to `@timestamp`.
- Click **Save data view to Kibana**.

9.2 Finding PetClinic Logs in Discover

- Go to **Discover** and select the `petclinic-*` Data View.
- Add a filter: `kubernetes.namespace_name is petclinic`.
- Adjust the time range to *Last 15 minutes* or *Today*.

This shows all log lines from pods running in the petclinic namespace.

9.3 Saving the PetClinic Errors View

To create a persistent saved search for application errors:

- In Discover with the `petclinic-*` Data View active, enter this query in the KQL search bar:

```
kubernetes.namespace_name : "petclinic" AND log : "*ERROR*"
```

- Confirm the results show only error-level log lines from the petclinic namespace.
- Click **Save** (top right), enter the name **PetClinic Errors**, and save.

The saved search is available from the Saved Objects list and can be reopened directly or embedded in a Dashboard.

Note on log : `"*ERROR*"` matching

Spring Boot plain-text logs include the word ERROR in uppercase as the log level token. The wildcard query matches any log line containing that string. This is intentionally broad — it also matches lines like 'error count: 0'. For a tighter match, a regex query or a structured log format with a dedicated level field would be more precise.

10. Troubleshooting Log

This section documents the specific problems encountered during the session and the exact fixes applied.

Problem: Elasticsearch Pod OOMKilled

Symptom: The pod starts, briefly reaches Running, then is killed and restarted. `kubectl describe pod` shows *OOMKilled* as the last termination reason.

Cause: Default Elasticsearch JVM settings allocate a large fraction of available memory. On a local single-node cluster, this exceeds what is available.

Fix: Add `ES_JAVA_OPTS=-Xms256m -Xmx256m` to the StatefulSet env block. This pins the heap to a fixed 256 MB on both sides.

```
# In the StatefulSet YAML, under env:
- name: ES_JAVA_OPTS
  value: "-Xms256m -Xmx256m"

kubectl apply -f 02-statefulset.yml -n logging
kubectl rollout restart statefulset/elastic-logging -n logging
```

Problem: Fluent Bit Rejected by Elasticsearch — Type Error

Symptom: Fluent Bit logs show HTTP 400 responses from Elasticsearch with a message about *type* not being allowed.

Cause: Elasticsearch 8.x removed document types. Fluent Bit's ES output plugin, by default, sends a `_type` field in bulk requests. ES 8.x rejects any request containing this field.

Fix: Add `Suppress_Type_Name On` to the `[OUTPUT]` block in `fluent-bit.conf`.

```
# In fluent-bit.conf OUTPUT section:
Suppress_Type_Name On

# Then apply and restart:
kubectl apply -f 10-fluentbit-configmap.yml -n logging
kubectl rollout restart -n logging ds/fluentbit
```

Problem: kubectl port-forward Fails on Port 5601 (Windows)

Symptom: Running `kubectl -n logging port-forward svc/kibana 5601:5601` fails with a bind error or silently exits.

Cause: Hyper-V on Windows reserves certain dynamic port ranges. Port 5601 is within one of these excluded ranges and cannot be bound by user processes.

Fix: Use a different local port that is not in any excluded range. Check with `netsh`, then use a clear port:

```
netsh interface ipv4 show excludedportrange protocol=tcp
# Identify a port NOT in any listed range, e.g. 9300

kubectl -n logging port-forward svc/kibana 9300:5601
```

Problem: TLS / HTTPS — Certificate SAN Mismatch

Symptom: Kibana cannot connect to Elasticsearch when the host is set to the Service name (*elasticsearch.logging.svc*). TLS verification fails with a hostname mismatch error.

Cause: Elasticsearch 8.x auto-generates a self-signed certificate. The Subject Alternative Names in that certificate include the pod hostname (*elastic-logging-0*) and *localhost*, but not the Kubernetes Service DNS name.

Paths explored:

- Using the pod hostname directly as the host: partially works but is fragile (pod hostnames can change with StatefulSet recreation).
- Extracting the CA certificate and mounting it into Kibana: requires additional Secret management and a specific `ELASTICSEARCH_SSL_CERTIFICATEAUTHORITIES` env var.
- Regenerating the certificate with additional SANs: requires modifying ES internal config before first start.

Resolution for the lab: TLS was disabled on the Elasticsearch HTTP layer by adding `xpack.security.http.ssl.enabled=false` to the StatefulSet env. All connections switched to plain HTTP. Authentication remains active.

```
# In the StatefulSet YAML, under env:
- name: xpack.security.http.ssl.enabled
  value: "false"

# Also update ELASTICSEARCH_HOSTS in Kibana Deployment from https to http:
# ELASTICSEARCH_HOSTS: '["http://elasticsearch.logging.svc:9200"]'
```

Problem: Kibana Shows 'degraded' Status

Symptom: The `/api/status` endpoint returns a level other than *available*. The Kibana UI loads but shows connection warnings.

Common cause: Kibana is still connecting to Elasticsearch, or the Elasticsearch hosts URL is wrong (e.g. still using https after switching to http).

Fix: Check Kibana logs for the specific error, correct the environment variable or ConfigMap, and restart the Deployment:

```
kubectl logs -n logging deploy/kibana --tail=80
# Identify the error, fix the ConfigMap or Secret
kubectl rollout restart deploy/kibana -n logging
```

Problem: No Indices Created in Elasticsearch

Symptom: `_cat/indices` shows no *petclinic*-* indices even after the DaemonSet has been running for several minutes.

Diagnostic steps:

- Check Fluent Bit logs for HTTP errors or authentication failures.
- Confirm the Elasticsearch Service is reachable from inside the cluster: run a debug pod and curl the Service.
- Verify the fluentbit user exists and has the correct role assigned.

```
kubectl logs -n logging -l app=fluentbit --tail=50
```

```
# Check user exists:
```

```
curl -u "elastic:<PASSWORD>" http://localhost:9200/_security/user/fluentbit
```

11. Quick Reference

Access Kibana

```
kubectl -n logging port-forward svc/kibana 9300:5601
# Browser: http://localhost:9300
# Login: elastic / <ES_SUPERUSER_PASSWORD>
```

Access Elasticsearch API

```
kubectl -n logging port-forward svc/elasticsearch 9200:9200
curl -u "elastic:<PASSWORD>" http://localhost:9200
```

List Indices

```
curl -u "elastic:<PASSWORD>" "http://localhost:9200/_cat/indices?v"
```

Count Documents

```
curl -u "elastic:<PASSWORD>" "http://localhost:9200/petclinic-*/_count?pretty"
```

View Fluent Bit Logs

```
kubectl logs -n logging -l app=fluentbit --tail=100
```

View Kibana Logs

```
kubectl logs -n logging deploy/kibana --tail=100
```

View Elasticsearch Logs

```
kubectl logs -n logging elastic-logging-0 --tail=100
```

List All Resources in Logging Namespace

```
kubectl get all -n logging
```

List All Secrets

```
kubectl get secrets -n logging
```

Restart After a Config Change

```
# Kibana
kubectl rollout restart -n logging deploy/kibana

# Fluent Bit (DaemonSet)
kubectl rollout restart -n logging ds/fluentbit

# Elasticsearch (StatefulSet – causes data pod restart, brief downtime)
kubectl rollout restart -n logging statefulset/elastic-logging
```

Apply All Manifests (Full Deploy, in Order)

```
kubectl apply -f 01-namespace.yml
kubectl apply -f 02-statefulset.yml -n logging
kubectl apply -f 03-service.yml -n logging
kubectl apply -f 04-kibana-deploy.yml -n logging
kubectl apply -f 05-kibana-configmap.yml -n logging
kubectl apply -f 06-kibana-service.yml -n logging
kubectl apply -f 07-fluentbit-sa.yml -n logging
kubectl apply -f 08-fluentbit-crole.yml
kubectl apply -f 09-fluentbit-crolebind.yml
kubectl apply -f 10-fluentbit-configmap.yml -n logging
kubectl apply -f 11-fluentbit-daemonset.yml -n logging
```

PetClinic Errors KQL Query (Kibana Discover)

```
kubernetes.namespace_name : "petclinic" AND log : "**ERROR**"
```

12. Known Gaps and Next Steps

12.1 Re-enable TLS on Elasticsearch

The current setup uses plain HTTP. To harden it:

- Remove `xpack.security.http.ssl.enabled=false` from the StatefulSet.
- After restart, extract the new auto-generated CA certificate from the pod.
- Store the CA in a Secret and mount it into both Kibana and Fluent Bit pods.
- Update Kibana's ELASTICSEARCH_HOSTS back to `https://`.
- Update Fluent Bit's output block with `tls On, tls.ca_file /path/to/http_ca.crt`.

The core constraint is that the auto-generated cert's SAN includes `elastic-logging-0` (the pod hostname) and `localhost`, but not the Service DNS name. Use the pod hostname in the hosts config for Kibana, and the Service name in Fluent Bit's Host setting with TLS verification disabled or a custom cert with the correct SANs.

12.2 Request ID Tracing

PetClinic logs are currently unstructured plain text with no request correlation. To enable per-request tracing:

- Add a servlet filter to PetClinic that generates a UUID per request and stores it in the SLF4J MDC (Mapped Diagnostic Context).
- Update the log pattern in `logback-spring.xml` to include the MDC value, e.g. `[requestId=%X{requestId}]`.
- Add a Fluent Bit Lua or regex filter to extract the request ID into a dedicated field.
- Search in Kibana: `request_id : "<uuid>"`.

12.3 Namespace Filtering in Fluent Bit

Currently Fluent Bit forwards logs from all namespaces (including its own debug output, kube-system, etc.), contributing to the high document count. Add a grep filter to limit forwarding to the petclinic namespace only:

```
[FILTER]
Name grep
Match kube.*
Regex $kubernetes['namespace_name'] petclinic
```

12.4 Multiline Stack Trace Merging

Spring Boot exception stack traces produce one log event per line. Fluent Bit's multiline parser can detect Java exception patterns and merge continuation lines into a single event, making stack traces readable as one document in Kibana.

12.5 Index Lifecycle Management

Daily indices accumulate indefinitely. An ILM policy can automatically roll over large indices and delete old ones, preventing unbounded disk growth.

12.6 Replace Basic Auth with API Key for Fluent Bit

Elasticsearch API keys are a more production-appropriate credential type. They can be scoped to specific index patterns, have expiry times, and can be revoked without changing a user's password.

Appendix. All Kubernetes Resources Created

Resource Type	Name	Namespace	Purpose
Namespace	logging	—	Isolates all logging infra (01-namespace.yml)
StatefulSet	elastic-logging	logging	Runs Elasticsearch single-node
Service (ClusterIP)	elasticsearch	logging	Internal ES access on port 9200
Deployment	kibana	logging	Runs Kibana web UI
Service (ClusterIP)	kibana	logging	Internal Kibana access on port 5601
DaemonSet	fluentbit	logging	One log collector per node
ServiceAccount	fluentbit	logging	Identity for Fluent Bit pods
ClusterRole	fluentbit	—	Read-only access to pods/namespaces API
ClusterRoleBinding	fluentbit	—	Binds ClusterRole to ServiceAccount
ConfigMap	kibana-config	logging	kibana.yml (server config)
ConfigMap	fluentbit-configmap	logging	fluent-bit.conf + parsers.conf
Secret	es-auth	logging	elastic superuser credentials
Secret	fluentbit-es-auth	logging	Fluent Bit writer credentials
Secret	kibana-es-token	logging	Kibana service account token
Secret	kibana-enc-keys	logging	Kibana encryption keys (3 values)
PVC	data-elastic-logging-0	logging	Elasticsearch data volume (1 Gi)